

Plan de Proyecto NestJS Multi-Rama

<https://gemini.google.com/app/814eac7965f7a134>

User prompt: dame los requerimientos del proyecto historias de usuario casos de uso dame un paso a paso para solucionarlo con buenas practicas se va a trabajar con antigravity y con nest new, dame las instrucciones despues de haber subido a git un repo con un nest ya creado y 6 ramas una de main y 5 ramas de integrantes el cual cada uno va a tener una parte dame todo porfis cada uno debe trabajar en su rama para despues hacer pr al main

Response: ¡Qué más, parceró! De una, vamos a meterle toda la ficha a este camello para que coronen ese proyecto. Póngale cuidado a cómo vamos a estructurar todo basándonos en el documento "Caso Práctico 5.pdf".

Requerimientos del Proyecto "TaskFlow Enterprise"

Requerimientos Funcionales

- **Gestión de Departamentos:** Crear y registrar las áreas de la empresa (ej: Desarrollo, Diseño, QA, Marketing).
- **Control de Empleados:** Administrar la información de los trabajadores y vincularlos a su respectivo departamento.
- **Registro de Proyectos:** Poder crear proyectos con un nombre, fecha de inicio y el departamento que se hace responsable.
- **Gestión de Tareas:** Crear tareas amarradas a un proyecto, asignarles un empleado y controlar su estado (Pendiente, En Progreso, Completada).
- **Lógica de Negocio:** Calcular de forma automática el porcentaje de avance del proyecto según las tareas terminadas frente al total de tareas.
- **Consultas Avanzadas:** Sacar el detalle completo de un proyecto que muestre el departamento, las tareas, los empleados encargados y el porcentaje de progreso que se calculó.

Requerimientos Técnicos

- **Framework:** NestJS manejando módulos, controladores y servicios.
- **Base de Datos:** MySQL utilizando el ORM TypeORM.
- **Seguridad:** Manejo de credenciales de conexión a través de variables de entorno (.env).
- **Estructura de Datos:** Uso estricto de DTOs para la entrada y actualización de datos.
- **Arquitectura:** Correcta inyección de dependencias con repositorios y servicios.

Historias de Usuario

- **HU1 - Departamentos:** Como administrador del sistema, quiero registrar y gestionar los departamentos de la agencia para organizar a los equipos de trabajo.

- **HU2 - Empleados:** Como administrador, quiero registrar a los empleados y asignarlos a un departamento para saber a qué área pertenecen.
- **HU3 - Proyectos:** Como líder, quiero crear proyectos asignándoles un nombre, fecha de inicio y departamento responsable para llevar un control del portafolio.
- **HU4 - Tareas:** Como miembro del equipo, quiero crear tareas dentro de un proyecto, asignar a un responsable y actualizar su estado para llevar el seguimiento de los entregables.
- **HU5 - Reporte de Avance:** Como gerente, quiero ver el detalle de un proyecto, incluyendo el equipo y el porcentaje de progreso automático, para evaluar el estado real del desarrollo.

Casos de Uso

1. **CU01 - Configurar Entorno:** Configurar la conexión a MySQL con TypeORM y el archivo `.env`.
2. **CU02 - Administrar Departamentos:** CRUD (Crear, Leer, Actualizar, Borrar) de la entidad Departamento.
3. **CU03 - Administrar Empleados:** CRUD de la entidad Empleado (con relación Muchos a Uno con Departamento).
4. **CU04 - Administrar Proyectos:** CRUD de Proyectos (con relación Muchos a Uno con Departamento).
5. **CU05 - Administrar Tareas:** CRUD de Tareas (Relación Muchos a Uno con Proyecto y Muchos a Uno con Empleado).
6. **CU06 - Consultar Detalle de Proyecto:** Endpoint tipo GET que retorna el proyecto con todas sus relaciones anidadas y calcula matemáticamente el % de progreso.

Paso a Paso y Buenas Prácticas (Flujo en AntiGravity, NestJS y Git)

Como ya tienen el cascarón armado con `nest new`, subido a Git y con las 6 ramas listas (la `main` y las 5 de los parceros), así es como deben camellar sin pisarse las mangueras:

1. Distribución del Trabajo (Las 5 Ramas)

Para que nadie toque el código del otro, repártanse el proyecto así:

- **Integrante 1 (Rama 1):** Configuración inicial (Instalar TypeORM, MySQL, `@nestjs/config` para el `.env`) y módulo de **Departamentos** (Entity, DTO, Service, Controller).
- **Integrante 2 (Rama 2):** Módulo de **Empleados** (Entity con relación a Departamento, DTO, Service, Controller).
- **Integrante 3 (Rama 3):** Módulo de **Proyectos** (Entity con relación a Departamento, DTO, Service, Controller).
- **Integrante 4 (Rama 4):** Módulo de **Tareas** (Entity con relación a Proyecto y Empleado, enum para los estados, DTO, Service, Controller).

- **Integrante 5 (Rama 5):** Lógica pesada. Endpoint de **Consultas Avanzadas** en el módulo de proyectos para calcular el porcentaje y traer toda la data relacionada.

2. Flujo de Trabajo Diario (Paso a Paso)

Cada parcero debe abrir su entorno en AntiGravity y seguir este flujo juicioso:

1. Traer lo más reciente (Siempre):

```
git checkout main
git pull origin main
```

2. Pasarse a su rama de trabajo:

```
git checkout rama-integrante-X
# Si necesitan actualizar su rama con los cambios nuevos de main:
git merge main
```

3. Desarrollo Rápido con Nest:

Para no escribir todo a mano, usen los generadores de NestJS. Por ejemplo, el parcero de Empleados escribe:

```
nest g resource empleados
```

(Esto les crea automáticamente el módulo, controlador, servicio y DTOs bien ordenaditos).

4. Guardar los cambios (Buenas Prácticas de Commits):

Nada de "cambios" o "ahora si sirve". Usen *Conventional Commits*:

```
git add .
git commit -m "feat(empleados): creacion de entidad y dtos de empleado"
```

5. Subir la rama:

```
git push origin rama-integrante-X
```

3. El Famoso Pull Request (PR) al Main

Cuando un parcero termina su parte y ya probó en su AntiGravity que todo compila bien:

1. Va a GitHub/GitLab.
2. Busca el botón verde que dice **"Compare & pull request"**.
3. Selecciona como base la rama `main` y como comparativa su rama.
4. **Buenas Prácticas:** Nunca hagan el merge ustedes mismos. Pídanle a otro compañero que revise el código y sea él quien le dé aprobar al PR. Así evitan que se rompa la aplicación principal.
5. Una vez integrado (Merged), los demás compañeros deben hacer `git checkout main` y `git pull` para tener la actualización antes de seguir camellando.

¡Hágale con toda que ese proyecto les va a quedar bien bacano! Me avisa si alguna de las partes del código se pone pesada y la revisamos.
